

types

TIME_UNITS module-attribute

```
TIME_UNITS = {(toString()) for value in (values())}
```

TimeUnit module-attribute

```
TimeUnit = autoclass('com.feedzai.commons.util.time.TimeUnit')
```

CategoricalField

```
CategoricalField(possible\_values: Optional[Set[str]] = None)
```

Bases: [DataScienceField](#)

Representation of a Pulse categorical field.

Parameters:

Name	Type	Description	Default
possible_values 	Set of str	Possible value for the categorical field.	None

[possible_values](#) instance-attribute

```
possible\_values = possible\_values
```

[subclasses](#) class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

[auto_cast](#) staticmethod

```
auto\_cast(java_object) -> CategoricalField
```

Helper method called when a return obj comes from java to be auto converted to python.

ComplexField

```
ComplexField()
```

Bases: [DataScienceField](#)

Representation of a Pulse ComplexField

[subclasses](#) class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

[auto_cast](#) staticmethod

```
auto\_cast(_) -> ComplexField
```

Helper method called when a return obj comes from java to be auto converted to python.

DataScienceField [↗](#)

```
DataScienceField(data\_science\_field)
```

Bases: [JavaWrapper](#)

Base class for every type.

Default constructor.

Parameters:

Name	Type	Description	Default
data_science_field ↗		Pulse's DataScienceField class.	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

DateTimeField [↗](#)

```
DateTimeField(time\_format: Format)
```

Bases: [DataScienceField](#)

Representation of a Pulse DateTimeField.

Parameters:

Name	Type	Description	Default
time_format ↗	Format	The time format for the date time field.	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

time_format **instance-attribute** [↗](#)

```
time_format = time\_format
```

auto_cast **staticmethod** [↗](#)

```
auto_cast(java_object) -> DateTimeField
```

Helper method called when a return obj comes from java to be auto converted to python.

DateTimeFormat [↗](#)

```
DateTimeFormat(time\_format: str)
```

Bases: [Format](#)

Representation of a date time format (equivalent to Pulse's DateTimeFormat).

Parameters:

Name	Type	Description	Default
<code>time_format</code> ¶	<code>str</code>	The date time pattern associated with this format.	<i>required</i>

subclasses `class-attribute` `instance-attribute` [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast `staticmethod` [¶](#)

```
auto_cast(java_object) -> DateFormat
```

Helper method called when a return obj comes from java to be auto converted to python.

validate [¶](#)

```
validate()
```

Format [¶](#)

```
Format(time\_format: JavaClass)
```

Bases: `JavaWrapper`

Base class for every format.

Parameters:

Name	Type	Description	Default
<code>time_format</code> ¶	<code>JavaClass</code>	The time format for the date time field.	<i>required</i>

subclasses `class-attribute` `instance-attribute` [¶](#)

```
subclasses: List[JavaWrapper] = []
```

NativeType [¶](#)

Bases: `Enum`

Supported Native Types in the creation of schema fields

BOOLEAN `class-attribute` `instance-attribute` [¶](#)

```
BOOLEAN = 'BOOLEAN'
```

DOUBLE `class-attribute` `instance-attribute` [¶](#)

```
DOUBLE = 'DOUBLE'
```

FLOAT `class-attribute` `instance-attribute` [¶](#)

```
FLOAT = 'FLOAT'
```

INT `class-attribute` `instance-attribute` [¶](#)

```
INT = 'INT'
```

LONG [class-attribute](#) [instance-attribute](#) [¶](#)

```
LONG = 'LONG'
```

STRING [class-attribute](#) [instance-attribute](#) [¶](#)

```
STRING = 'STRING'
```

UTF8STRING [class-attribute](#) [instance-attribute](#) [¶](#)

```
UTF8STRING = 'UTF8STRING'
```

NumericalField [¶](#)

```
NumericalField()
```

Bases: [DataScienceField](#)

Representation of a Pulse NumericalField

subclasses [class-attribute](#) [instance-attribute](#) [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast [staticmethod](#) [¶](#)

```
auto_cast(_) -> NumericalField
```

Helper method called when a return obj comes from java to be auto converted to python.

StringField [¶](#)

```
StringField()
```

Bases: [DataScienceField](#)

Representation of a Pulse StringField

subclasses [class-attribute](#) [instance-attribute](#) [¶](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast [staticmethod](#) [¶](#)

```
auto_cast(_) -> StringField
```

Helper method called when a return obj comes from java to be auto converted to python.

TimeModel [¶](#)

```
TimeModel(value: int, unit: TimeUnit)
```

Bases: [JavaWrapper](#)

Representation of a Pulse TimeModel to construct a time window.

Parameters:

Name	Type	Description	Default
value ↗	int	Value of the time window.	<i>required</i>
unit ↗	TimeUnit	Enum object to define the window, possible values: MILLISECOND SECOND MINUTE HOUR DAY WEEK	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast **staticmethod** [↗](#)

```
auto_cast(java_object) -> TimeModel
```

Helper method called when a return obj comes from java to be auto converted to python.

TimestampFormat [↗](#)

```
TimestampFormat(unit: str)
```

Bases: [Format](#)

Representation of a timestamp format (equivalent to Pulse's TimestampFormat).

Parameters:

Name	Type	Description	Default
unit ↗	str	The time format identifier. The domain values are (the java.concurrent.TimeUnit values): NANSECONDS, MICROSECONDS, MILLISECONDS, SECONDS, MINUTES, HOURS, DAYS.	<i>required</i>

subclasses **class-attribute** **instance-attribute** [↗](#)

```
subclasses: List[JavaWrapper] = []
```

auto_cast **staticmethod** [↗](#)

```
auto_cast(java_object) -> TimestampFormat
```

Helper method called when a return obj comes from java to be auto converted to python.

get_dtype [↗](#)

```
get_dtype(ds_field, pyspark_context: bool = False) -> DataScienceField
```

Get the python representation of a DataScienceField from a Pulse field.

Parameters:

Name	Type	Description	Default
ds_field ↗			<i>required</i>

Name	Type	Description	Default
<code>pyspark_context</code> 	boolean	Indicates whether this is a pyspark context or a Pulse context. In a pyspark context the type of time fields will be given as timestamp in seconds (this is the default unit after the pulse_to_pyspark transformation), otherwise the actual type from Pulse will be returned.	False

Returns:

Type	Description
<code>DataScienceField</code>	The corresponding python DataScienceField representation.

`get_time_format`

```
get_time_format(ds_field) -> Format
```

Get the python TimeFormat representation from a Pulse field.

Parameters:

Name	Type	Description	Default
<code>ds_field</code> 		A Pulse DataScienceField corresponding to a DateTimeField.	<i>required</i>

Returns:

Type	Description
<code>Format</code>	The associated time Format.